

1. Configure 2 slave machines in the Jenkins master.

Approch-1

Task 1 — Configure 2 Jenkins Agent Machines Using Dedicated SSH Key

Objective

Configure one Jenkins Controller and two Jenkins Agent EC2 instances and establish SSH-based communication between the Controller and both Agents.

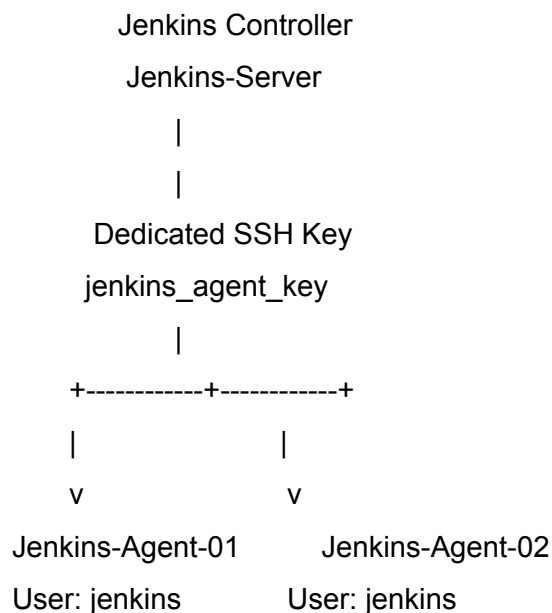
A dedicated Linux user named:

jenkins

is used for Jenkins agent communication.

A dedicated SSH key pair is generated on the **Jenkins Controller** specifically for Controller-to-Agent communication.

Architecture



Java 21

/home/jenkins/agent

Java 21

/home/jenkins/agent

Current Jenkins terminology:

Master → Controller

Slave → Agent

Purpose

Jenkins uses a distributed architecture where:

Controller

→ Manages jobs

→ Schedules builds

→ Stores Jenkins configuration

→ Controls agents

Agents

→ Execute jobs

→ Compile/build applications

→ Run tests

→ Run deployment commands

Using dedicated agents reduces workload on the Jenkins Controller.

Using a dedicated SSH key provides separate authentication specifically for:

Jenkins Controller → Jenkins Agents

Prerequisites

Create three Amazon Linux 2023 EC2 instances:

Jenkins-Server

Jenkins-Agent-01

Jenkins-Agent-02

Requirements:

Same VPC

Port 22 allowed between Controller and Agents

Java installed on Agents

Jenkins installed on Controller

Useful verification commands:

```
free -m
```

Checks memory.

```
cat /proc/cpuinfo
```

Checks CPU information.

```
cat /etc/os-release
```

Checks operating system information.

Step 1 — Verify Jenkins User on Controller

After Jenkins installation, first verify whether the Jenkins user already exists:

```
id jenkins
```

Normally Jenkins package installation creates the `jenkins` service user automatically.

Also verify its home directory:

```
getent passwd jenkins
```

Do **not** create another `jenkins` user if it already exists.

Step 2 — Create Jenkins User on Agent-01

Connect:

```
ssh -i "my_server.pem" ec2-user@<AGENT-01-PUBLIC-IP>
```

Create dedicated user:

```
sudo useradd -m -s /bin/bash jenkins
```

Verify:

```
id jenkins
```

Check home directory:

```
sudo ls -ld /home/jenkins
```

Expected:

```
jenkins jenkins /home/jenkins
```

Step 3 — Create Jenkins User on Agent-02

Connect:

```
ssh -i "my_server.pem" ec2-user@<AGENT-02-PUBLIC-IP>
```

Create:

```
sudo useradd -m -s /bin/bash jenkins
```

Verify:

```
id jenkins
```

Check:

```
sudo ls -ld /home/jenkins
```

Step 4 — Install Java 21 on Both Agents

On **Agent-01** and **Agent-02**, run:

```
sudo dnf install java-21-amazon-corretto-devel -y
```

Verify:

```
java -version
```

Expected:

```
openjdk version "21..."
Amazon Corretto
```

Locate Java:

```
readlink -f $(which java)
```

Jenkins Agents require Java because Jenkins Remoting runs as a Java process.

Step 5 — Prepare SSH Directory on Agent-01

Run:

```
sudo mkdir -p /home/jenkins/.ssh
```

Set ownership:

```
sudo chown jenkins:jenkins /home/jenkins/.ssh
```

Set permission:

```
sudo chmod 700 /home/jenkins/.ssh
```

Verify:

```
sudo ls -ld /home/jenkins/.ssh
```

Expected:

```
drwx----- jenkins jenkins /home/jenkins/.ssh
```

Repeat the same commands on **Agent-02**.

Step 6 — Generate Dedicated SSH Key on Jenkins Controller

Connect to Jenkins Controller.

Verify Jenkins user:

```
id jenkins
```

Switch to Jenkins user.

If normal login shell works:

```
sudo -iu jenkins
```

If it does not, use:

```
sudo -u jenkins -H bash
```

Verify:

```
whoami
```

Expected:

```
jenkins
```

Create SSH directory:

```
mkdir -p ~/.ssh
```

Set permission:

```
chmod 700 ~/.ssh
```

Generate dedicated SSH key:

```
ssh-keygen -t rsa -b 4096 -m PEM \
-f ~/.ssh/jenkins_agent_key
```

When prompted for passphrase, for this lab press:

Enter

Enter

This creates:

```
jenkins_agent_key
jenkins_agent_key.pub
```

Meaning:

```
jenkins_agent_key
→ Private Key
```

```
jenkins_agent_key.pub
→ Public Key
```

Verify:

```
ls -la ~/.ssh
```

Step 7 — Set SSH Key Permissions

Set private-key permissions:

```
chmod 600 ~/.ssh/jenkins_agent_key
```

Set public-key permissions:

```
chmod 644 ~/.ssh/jenkins_agent_key.pub
```

Verify:

```
ls -l ~/.ssh/jenkins_agent_key
ls -l ~/.ssh/jenkins_agent_key.pub
```

Security concept:

Private Key

- Remains on Jenkins Controller
- Must never be shared publicly

Public Key

- Copied to Jenkins Agents
-

Step 8 — Copy Public Key

On Jenkins Controller:

```
cat ~/.ssh/jenkins_agent_key.pub
```

Copy the entire output.

It will look similar to:

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAQ...
```

Step 9 — Add Public Key to Agent-01

Connect to Agent-01.

Create:

```
sudo touch /home/jenkins/.ssh/authorized_keys
```

Edit:

```
sudo vi /home/jenkins/.ssh/authorized_keys
```

Paste the **public key** generated on Jenkins Controller.

Save:

Esc
:wq
Enter

Set ownership:

```
sudo chown jenkins:jenkins \  
/home/jenkins/.ssh/authorized_keys
```

Set permission:

```
sudo chmod 600 \  
/home/jenkins/.ssh/authorized_keys
```

Verify:

```
sudo ls -l /home/jenkins/.ssh/authorized_keys
```

Step 10 — Add Public Key to Agent-02

Perform the same steps on Agent-02:

```
sudo touch /home/jenkins/.ssh/authorized_keys
```

Edit:

```
sudo vi /home/jenkins/.ssh/authorized_keys
```

Paste the same Controller public key.

Then:

```
sudo chown jenkins:jenkins \  
/home/jenkins/.ssh/authorized_keys  
sudo chmod 600 \  
/home/jenkins/.ssh/authorized_keys
```

Step 11 — Verify SSH Configuration on Agents

On both agents run:

```
sudo sshd -T | grep -E \  
'pubkeyauthentication|authorizedkeysfile'
```

Verify SSH service:

```
sudo systemctl status sshd
```

If required:

```
sudo systemctl restart sshd
```

SSH public-key authentication should be enabled.

Step 12 — Test Controller → Agent-01 SSH

On Jenkins Controller, as `jenkins` user:

```
ssh -o IdentitiesOnly=yes \  
-i ~/.ssh/jenkins_agent_key \  
jenkins@<AGENT-01-PRIVATE-IP>
```

Using **private IP** is preferable when Controller and Agents are in the same VPC.

Verify:

```
whoami
```

Expected:

```
jenkins
```

Then:

```
pwd
```

Expected:

```
/home/jenkins
```

Check Java:

```
java -version
```

Exit:

```
exit
```

Step 13 — Test Controller → Agent-02 SSH

From Controller:

```
ssh -o IdentitiesOnly=yes \  
-i ~/.ssh/jenkins_agent_key \  
jenkins@<AGENT-02-PRIVATE-IP>
```

Verify:

```
whoami  
pwd  
java -version
```

Successful SSH from Controller to both Agents proves the dedicated SSH configuration is working.

Step 14 — Create Agent Workspace

On **Agent-01** and **Agent-02**:

```
sudo mkdir -p /home/jenkins/agent
```

Set ownership:

```
sudo chown -R jenkins:jenkins /home/jenkins/agent
```

Set permissions:

```
sudo chmod 755 /home/jenkins/agent
```

Verify:

```
ls -ld /home/jenkins/agent
```

This directory will be used by Jenkins for:

remoting.jar

workspace/

build files

Step 15 — Add Dedicated Private Key to Jenkins Credentials

Open:

Jenkins

→ Manage Jenkins

→ Credentials

→ System

→ Global credentials

→ Add Credentials

Configure:

Kind:

SSH Username with private key

Scope:

Global

Username:

jenkins

Private Key:

Enter directly

ID:

jenkins-agent-ssh

Description:

Dedicated Jenkins Agent SSH Key

On Jenkins Controller, display private key:

```
sudo -u jenkins cat \  
~jenkins/.ssh/jenkins_agent_key
```

Alternatively, while already logged in as `jenkins`:

```
cat ~/.ssh/jenkins_agent_key
```

Copy the entire private key:

```
-----BEGIN RSA PRIVATE KEY-----  
...  
-----END RSA PRIVATE KEY-----
```

Paste it into Jenkins credentials.

Important

Use:

jenkins_agent_key

Do **not** use:

jenkins_agent_key.pub

Jenkins needs the **private key**.

Do not expose the private key in screenshots or documentation.

Step 16 — Create Jenkins-Agent-01 Node

Go to:

Manage Jenkins

→ Nodes

→ New Node

Enter:

Node name:

Jenkins-Agent-01

Type:

Permanent Agent

Configure:

Number of executors:

1

If your trainer specifically requires 2, set:

2

Remote root directory:

/home/jenkins/agent

Labels:

agent01 linux

Usage:

For strict label-based execution:

Only build jobs with label expressions matching this node

Launch method:

Launch agents via SSH

Host:

<AGENT-01-PRIVATE-IP>

Credentials:

jenkins-agent-ssh

Host Key Verification Strategy:

Manually trusted key Verification Strategy

Save.

Step 17 — Trust Agent-01 SSH Host Key

Open:

Manage Jenkins

→ Nodes

→ Jenkins-Agent-01

Trust the SSH fingerprint when prompted.

Then:

Relaunch agent

Open:

Log

Successful output should include:

[SSH] Starting sftp client.

[SSH] Copying latest remoting.jar...

[SSH] Starting agent process...

This is a Unix agent

Agent successfully connected and online

Screenshot

Take screenshot showing:

Agent successfully connected and online

Caption:

Figure 1: Jenkins-Agent-01 successfully connected to Jenkins Controller using dedicated SSH authentication.

Step 18 — Create Jenkins-Agent-02 Node

Go to:

Manage Jenkins

→ Nodes

→ New Node

Configure:

Name:

Jenkins-Agent-02

Type:

Permanent Agent

Executors:

1

Remote root:

/home/jenkins/agent

Labels:

agent02 linux

Usage:

Only build jobs with label expressions matching this node

Launch method:

Launch agents via SSH

Host:

<AGENT-02-PRIVATE-IP>

Credentials:

jenkins-agent-ssh

Host Key Verification Strategy:

Manually trusted key Verification Strategy

Save.

Step 19 — Trust Agent-02 SSH Host Key

Open:

Manage Jenkins

- Nodes
- Jenkins-Agent-02

Trust fingerprint.

Then:

- Relaunch agent
- Log

Successful result:

Agent successfully connected and online

Step 20 — Final Verification

Go to:

- Manage Jenkins
- Nodes

Verify:

Built-In Node	Online
Jenkins-Agent-01	Online
Jenkins-Agent-02	Online

Connected agents should display information such as:

Architecture
Linux (amd64)

Clock Difference
In sync

Free Disk Space

Free Temp Space

Response Time

Screenshot

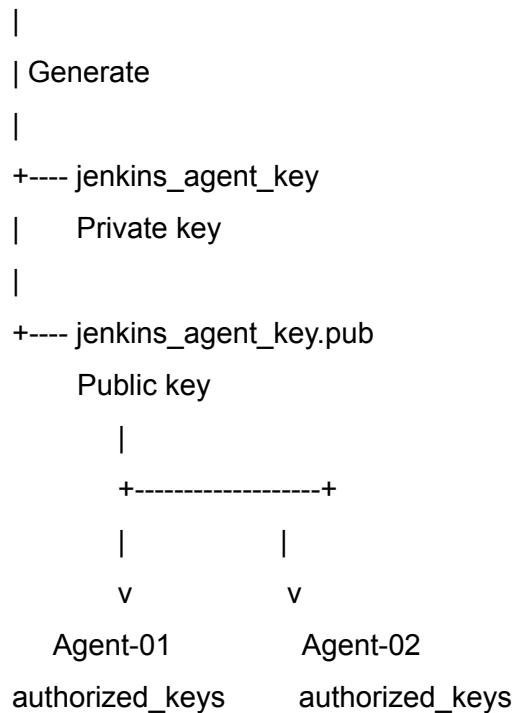
Take the Nodes page showing both Agents online.

Caption:

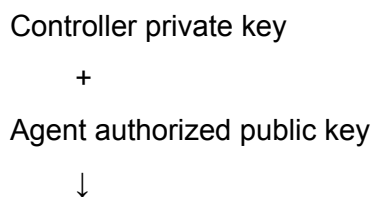
Figure 2: Two Jenkins Agents successfully connected to the Jenkins Controller using dedicated SSH authentication.

Dedicated SSH Authentication Flow

Jenkins Controller



During connection:



SSH authentication successful



Jenkins Agent starts

Jenkins User Security Concept

root

→ Full system privileges

ec2-user

→ Administrative login user

jenkins

→ Dedicated Jenkins execution user

Builds execute using:

jenkins

instead of:

root

This follows the **principle of least privilege**.

Troubleshooting

Problem 1 — Permission denied (publickey)

Possible reasons:

Wrong public key

Wrong private key

Wrong jenkins user

Incorrect .ssh permissions

Verify:

```
sudo ls -ld /home/jenkins/.ssh
sudo ls -l /home/jenkins/.ssh/authorized_keys
```

Required:

```
.ssh          700
authorized_keys 600
```

Owner:

```
jenkins:jenkins
```

Problem 2 — Known Hosts Error

Error:

```
No Known Hosts file was found
```

or:

```
Key exchange was not finished
```

Fix:

```
Agent
→ Configure
→ Host Key Verification Strategy
→ Manually trusted key Verification Strategy
```

Then trust the SSH fingerprint.

Problem 3 — Agent Shows N/A

If Jenkins shows:

```
Architecture  N/A
Disk Space    N/A
Response Time  N/A
```

the agent has not connected successfully.

Check:

Node

→ Log

Successful connection should show:

Agent successfully connected and online

Problem 4 — Java Not Found

Verify on agent:

```
java -version
```

Install if missing:

```
sudo dnf install java-21-amazon-corretto-devel -y
```

Task 1 Final Result

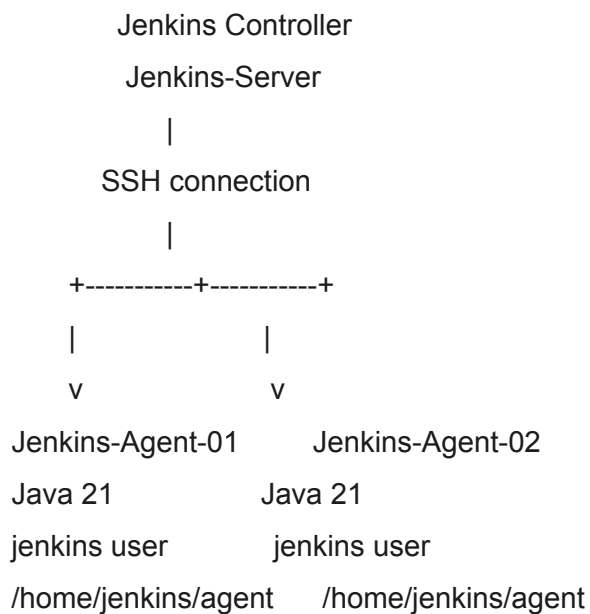
Jenkins Controller configured	✓
Two Agent EC2 instances created	✓
Dedicated jenkins users configured	✓
Java 21 installed on both Agents	✓
Dedicated SSH key generated	✓
Public key copied to both Agents	✓
Controller → Agent SSH tested	✓
Agent workspace created	✓
Private key added to Jenkins	✓
Agent-01 node configured	✓
Agent-02 node configured	✓
Agent-01 connected	✓
Agent-02 connected	✓
Both Agents online	✓

Task 1 — Configure 2 Jenkins Agent Machines

Objective

Configure two Jenkins agent machines and connect them to the Jenkins Controller using SSH.

Final architecture:



Current Jenkins terminology is:

Master → Controller

Slave → Agent

Step 1 — Create Two EC2 Agent Machines

In AWS:

EC2

→ Instances

→ Launch instances

Create two Amazon Linux 2023 instances.

Configuration used:

Names:

Jenkins-Agent-01

Jenkins-Agent-02

AMI:

Amazon Linux 2023

Instance type:

t3.micro

Key pair:

my_server

VPC:

Same VPC as Jenkins Controller

Public IP:

Enabled for lab

SSH:

Port 22 allowed

After launching, verify all three machines are running:

Jenkins-Server
Jenkins-Agent-01
Jenkins-Agent-02

Screenshot to add

Use the EC2 screenshot showing all three instances in **Running** state.

Caption:

Figure 1: Jenkins Controller and two Jenkins Agent EC2 instances running in AWS.

Step 2 — Connect to Agent-01

From Git Bash:

```
ssh -i "my_server.pem" ec2-user@<AGENT-01-PUBLIC-IP>
```

Become root temporarily for system configuration:

```
sudo su -
```

Important:

We used root only for installation and user creation.

Actual Jenkins agent work runs using:

```
jenkins
```

Step 3 — Install Java 21 on Agent-01

Run:

```
dnf install java-21-amazon-corretto-devel -y
```

Verify:

```
java -version
```

Result:

```
openjdk version "21.0.12.1"
```

```
Amazon Corretto
```

Jenkins agents require Java because Jenkins Remoting runs as a Java process.

Step 4 — Create Jenkins Linux User

Create a dedicated service account:

```
useradd -m -s /bin/bash jenkins
```

Verify:

```
id jenkins
```

Example result:

```
uid=1001(jenkins)
```

```
gid=1001(jenkins)
```

```
groups=1001(jenkins)
```

This is important because Jenkins builds should not normally run as **root**.

Concept:

```
root
```

→ Full system access

```
ec2-user
```

→ Login user with sudo access

```
jenkins
```

→ Restricted Jenkins service/agent user

Step 5 — Configure SSH Access for Jenkins User

Create `.ssh` directory:

```
mkdir -p /home/jenkins/.ssh
```

Copy the existing EC2 authorized key:

```
cp /home/ec2-user/.ssh/authorized_keys \  
/home/jenkins/.ssh/authorized_keys
```

Set ownership:

```
chown -R jenkins:jenkins /home/jenkins/.ssh
```

Set secure permissions:

```
chmod 700 /home/jenkins/.ssh  
chmod 600 /home/jenkins/.ssh/authorized_keys
```

This allows Jenkins Controller to SSH directly using:

```
Username: jenkins
```

Step 6 — Verify Jenkins User

Switch using:

```
sudo -iu jenkins
```

Verify:

```
whoami  
pwd  
java -version
```

Expected:

```
jenkins
```

```
/home/jenkins
```

```
openjdk version "21..."
```

Important note:

This command may fail:

```
su - jenkins
```

because no password was configured for the `jenkins` account.

Instead use:

```
sudo -iu jenkins
```

Step 7 — Test Direct SSH as Jenkins User

From your local machine:

```
ssh -i "my_server.pem" jenkins@<AGENT-01-PUBLIC-IP>
```

Verify:

```
whoami
```

```
pwd
```

```
java -version
```

Expected:

```
jenkins
```

```
/home/jenkins
```

```
Java 21
```

This confirms SSH ownership and permissions are configured correctly.

Step 8 — Create Jenkins Agent Workspace

While logged in as `jenkins`:

```
mkdir -p /home/jenkins/agent  
chmod 755 /home/jenkins/agent
```

Verify:

```
ls -ld /home/jenkins/agent
```

Owner should be:

```
jenkins jenkins
```

This directory becomes the Jenkins agent remote workspace.

Step 9 — Create SSH Credential in Jenkins

Go to:

```
Manage Jenkins  
→ Credentials  
→ System  
→ Global credentials  
→ Add Credentials
```

Choose:

Kind:
SSH Username with private key

Scope:
Global

Username:

jenkins

For Private Key:

Enter directly

Paste the complete contents of:

my_server.pem

Set:

ID:

jenkins-agent-ssh

Description:

Jenkins Agent SSH Key

Save the credential.

Important:

Do not expose the private key in screenshots.

Step 10 — Create Jenkins-Agent-01 Node

Go to:

Manage Jenkins

→ Nodes

→ New Node

Enter:

Node name:

Jenkins-Agent-01

Type:

Permanent Agent

Configure:

Number of executors:

1

Remote root directory:

/home/jenkins/agent

Labels:

agent01 linux

Usage:

Use this node as much as possible

Launch method:

Launch agents via SSH

For Host, use Agent-01 private IP:

172.31.12.196

Using the private IP is preferred because Controller and Agent are in the same VPC.

Credential:

jenkins-agent-ssh

Host Key Verification Strategy:

Manually trusted key Verification Strategy

Save.

Step 11 — Trust SSH Host Key

Open:

Manage Jenkins

→ Nodes

→ Jenkins-Agent-01

Click:

Trust SSH Host Key

Verify/trust the fingerprint.

Then click:

Relaunch agent

Open:

Log

Successful output included:

[SSH] Starting sftp client.

[SSH] Copying latest remoting.jar...

[SSH] Starting agent process...

This is a Unix agent

Agent successfully connected and online

Screenshot to add

Use Agent-01 Log showing:

Agent successfully connected and online

Caption:

Figure 2: Jenkins-Agent-01 successfully connected to Jenkins Controller using SSH.

Step 12 — Configure Agent-02

Repeat the same configuration on Agent-02.

Connect:

```
ssh -i "my_server.pem" ec2-user@<AGENT-02-PUBLIC-IP>
```

Install Java:

```
sudo su -
```

```
dnf install java-21-amazon-corretto-devel -y
```

Create Jenkins user:

```
useradd -m -s /bin/bash jenkins
```

Configure SSH:

```
mkdir -p /home/jenkins/.ssh
```

```
cp /home/ec2-user/.ssh/authorized_keys \  
/home/jenkins/.ssh/authorized_keys
```

```
chown -R jenkins:jenkins /home/jenkins/.ssh
```

```
chmod 700 /home/jenkins/.ssh
```

```
chmod 600 /home/jenkins/.ssh/authorized_keys
```

Switch:

```
sudo -iu jenkins
```

Create workspace:

```
mkdir -p /home/jenkins/agent  
chmod 755 /home/jenkins/agent
```

Verify:

```
whoami  
pwd  
java -version
```

Step 13 — Create Jenkins-Agent-02 Node

In Jenkins:

```
Manage Jenkins  
→ Nodes  
→ New Node
```

Configure:

Name:
Jenkins-Agent-02

Executors:
1

Remote root:
/home/jenkins/agent

Labels:
agent02 linux

Launch method:
Launch agents via SSH

Host:

<Agent-02 PRIVATE IP>

Credentials:

jenkins-agent-ssh

Issue Faced — Known Hosts Error

Initially Agent-02 failed with:

No Known Hosts file was found at:

/var/lib/jenkins/.ssh/known_hosts

and:

Key exchange was not finished,
connection is closed.

The Jenkins log showed:

KnownHostsFileKeyVerificationStrategy

This meant Jenkins was trying to verify the Agent using a **known_hosts** file that did not exist.

Fix

Open:

Jenkins-Agent-02

→ Configure

Change:

Known hosts file Verification Strategy

to:

Manually trusted key Verification Strategy

Save.

Then:

Trust SSH Host Key

→ Relaunch agent

→ Log

Final successful output:

[SSH] Starting sftp client.

[SSH] Remote file system root

/home/jenkins/agent does not exist.

Will try to create it...

[SSH] Copying latest remoting.jar...

Starting agent process...

This is a Unix agent

Agent successfully connected and online

Screenshot to add

Use your Agent-02 log screenshot showing:

This is a Unix agent

Agent successfully connected and online

Caption:

Figure 3: Jenkins-Agent-02 successfully connected after fixing SSH Host Key Verification Strategy.

Step 14 — Final Verification

Go to:

Manage Jenkins

→ Nodes

Verify:

Built-In Node Online

Jenkins-Agent-01 Online

Jenkins-Agent-02 Online

For both agents Jenkins should display values such as:

Architecture

Linux (amd64)

Clock Difference

In sync

Free Disk Space

Free Temp Space

Response Time

When these values appear instead of **N/A**, the agent is connected successfully.

Final screenshot to add

Take the Nodes page showing both:

Jenkins-Agent-01

Jenkins-Agent-02

online.

Caption:

Figure 4: Two Jenkins agents successfully connected and monitored by Jenkins Controller.

Jenkins User / Permission Concept

One important learning from this task is:

```
Jenkins Controller
|
| SSH
v
Agent machine
|
v
jenkins Linux user
|
+---- /home/jenkins
|
+---- /home/jenkins/agent
|
+---- remoting.jar
|
+---- Jenkins workspace
```

Builds run with the permissions of:

```
jenkins
```

rather than:

```
root
```

This provides better security through **least privilege**.

Troubleshooting Learned

Problem 1:

su - jenkins

→ Authentication failure

Reason:

No password configured.

Fix:

sudo -iu jenkins

Problem 2:

No known_hosts file

→ SSH key exchange failed

Reason:

KnownHostsFile verification strategy selected.

Fix:

Use Manually trusted key Verification Strategy.

Problem 3:

Agent showing N/A

Meaning:

Agent has not successfully connected.

Successful connection shows:

Architecture

Disk Space

Response Time

etc.

Task 1 Final Result

Jenkins Controller configured 

Jenkins-Agent-01 created 

Jenkins-Agent-02 created 

Java 21 installed on both agents 

jenkins Linux user created 

SSH permissions configured 

SSH login as jenkins tested 

Agent workspace configured 

SSH Jenkins credential configured 

Agent-01 connected 

Agent-02 connected 

Both agents online 

aws [Search] [Alt+S] United States (N. Virginia) chandrazhekar (986881030460) chandrazhekar

EC2 > Instances

Successfully initiated starting of i-0f04615f70475d59b

Instances (1/3) Info Last updated less than a minute ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Saved filter sets [Choose filter set](#)

☐	Name ↗	Instance ID	Instance state ↕	Instance type ↕	Status check ↕	Availability Zone ↕	Public IPv4 DNS ↕	Public IPv
☐	Jenkins-Server	i-0f04615f70475d59b	Running ↕	t3.micro	3/3 checks passed	us-east-1a	ec2-34-200-237-174.co...	34.200.23
☒	Jenkins-Agent-01	i-0362eccfd8da96d2c	Running ↕	t3.micro	3/3 checks passed	us-east-1a	ec2-3-235-162-186.co...	3.235.162
☐	Jenkins-Agent-02	i-04773c0511cdfa3da	Running ↕	t3.micro	3/3 checks passed	us-east-1a	ec2-3-231-31-171.com...	3.231.31.1

i-0362eccfd8da96d2c (Jenkins-Agent-01)

[Details](#) [Status and alarms](#) [Monitoring](#) [Security](#) [Networking](#) [Storage](#) [Tags](#)

▼ Instance summary Info

Instance ID i-0362eccfd8da96d2c	Public IPv4 address 3.235.162.186 open address	Private IPv4 addresses 172.31.12.196
IPv6 address -	Instance state Running	Public DNS ec2-3-235-162-186.compute-1.amazonaws.com open address
Hostname type IP name: ip-172-31-12-196.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-12-196.ec2.internal	Elastic IP addresses -
Answer private resource DNS name IPv4 (A)	Instance type t3.micro	

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Agent-01

jenkins@ip-172-31-12-196:~

```
Verifying      : dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch
Verifying      : fontconfig-2.13.94-2.amzn2023.0.2.x86_64
Verifying      : fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch
Verifying      : freetype-2.13.2-5.amzn2023.0.2.x86_64
Verifying      : google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
Verifying      : google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
Verifying      : graphite2-1.3.14-7.amzn2023.0.3.x86_64
Verifying      : harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
Verifying      : java-21-amazon-corretto-devel-1:21.0.12+9-1.amzn2023.1.x86_64
Verifying      : java-21-amazon-corretto-headless-1:21.0.12+9-1.amzn2023.1.x86_64
Verifying      : javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch
Verifying      : langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
Verifying      : libX11-1.8.10-2.amzn2023.0.1.x86_64
Verifying      : libX11-common-1.8.10-2.amzn2023.0.1.noarch
Verifying      : libXau-1.0.11-6.amzn2023.0.1.x86_64
Verifying      : libXext-1.3.6-1.amzn2023.0.1.x86_64
Verifying      : libXrender-0.9.11-6.amzn2023.0.1.x86_64
Verifying      : libbrotli-1.1.0-1.amzn2023.0.1.x86_64
Verifying      : libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
Verifying      : libpng-2:1.6.37-10.amzn2023.0.13.x86_64
Verifying      : libxcb-1.17.0-1.amzn2023.0.1.x86_64
Verifying      : pixman-0.43.4-1.amzn2023.0.4.x86_64
Verifying      : xml-common-0.6.3-56.amzn2023.0.2.noarch
```

Installed:

```
alsa-lib-1.2.7.2-1.amzn2023.0.4.x86_64
dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
java-21-amazon-corretto-devel-1:21.0.12+9-1.amzn2023.1.x86_64
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
libXau-1.0.11-6.amzn2023.0.1.x86_64
libbrotli-1.1.0-1.amzn2023.0.1.x86_64
libxcb-1.17.0-1.amzn2023.0.1.x86_64
cairo-1.18.0-4.amzn2023.0.3.x86_64
dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch
freetype-2.13.2-5.amzn2023.0.2.x86_64
graphite2-1.3.14-7.amzn2023.0.3.x86_64
java-21-amazon-corretto-headless-1:21.0.12+9-1.amzn2023.1.x86_64
libX11-1.8.10-2.amzn2023.0.1.x86_64
libXext-1.3.6-1.amzn2023.0.1.x86_64
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
pixman-0.43.4-1.amzn2023.0.4.x86_64
```

Complete!

```
[root@ip-172-31-12-196 ~]# java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
[root@ip-172-31-12-196 ~]# useradd -m -s /bin/bash jenkins
[root@ip-172-31-12-196 ~]# id jenkins
uid=1001(jenkins) gid=1001(jenkins) groups=1001(jenkins)
[root@ip-172-31-12-196 ~]# mkdir -p /home/jenkins/.ssh
cp /home/ec2-user/.ssh/authorized_keys /home/jenkins/.ssh/authorized_keys
[root@ip-172-31-12-196 ~]# chown -R jenkins:jenkins /home/jenkins/.ssh
[root@ip-172-31-12-196 ~]# chmod 700 /home/jenkins/.ssh
chmod 600 /home/jenkins/.ssh/authorized_keys
[root@ip-172-31-12-196 ~]# su - jenkins
[jenkins@ip-172-31-12-196 ~]$ whoami
pwd
java -version
jenkins
/home/jenkins
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
[jenkins@ip-172-31-12-196 ~]$ |
```


← → ↻ 🏠 ⚠ Not secure 34.200.237.174:8081/computer/Jenkins-Agent-01/log ☆

Apps 🗄 | YouTube Google 12 Beginner Python... Learn Python Learn Data Structur... LeetCode - The Wor... Movierulz | Watch B... Slack

 **Jenkins** / Nodes ▾ / Jenkins-Agent-01 ▾ / Log 🔍

```
{
  ( alias;
    eval ${which_declare} ) | /usr/bin/which --tty-only --read-alias --read-functions --show-tilde --show-dot "$@"
  }
[09/29/26 10:48:40] [SSH] Starting sftp client.
[09/29/26 10:48:40] [SSH] Copying latest remoting.jar...
[09/29/26 10:48:41] [SSH] Copied 1,407,938 bytes.
Expanded the channel window size to 4MB
[09/29/26 10:48:41] [SSH] Starting agent process: cd "/home/jenkins/agent" && java -jar remoting.jar -workDir /home/jenkins/agent -jar-
/home/jenkins/agent/remoting/jarCache
Sep 29, 2026 10:48:41 AM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using /home/jenkins/agent/remoting as a remoting work directory
Sep 29, 2026 10:48:41 AM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to /home/jenkins/agent/remoting
<===[JENKINS REMOTING CAPACITY]==>channel started
Remoting version: 3355.3357.v931d3c992987
Launcher: SSHLauncher
Communication Protocol: Standard in/out
This is a Unix agent
Agent successfully connected and online
```

← → ↻ 🏠 ⚠ Not secure 34.200.237.174:8081/computer/Jenkins-Agent-... ☆

Apps 🗄 | YouTube Google 12 Beginner Python... Learn Python >>

 / Nodes ▾ / Jenkins-Agent-02 ▾ / Log 🔍 ⚙️ 👤

```
}
[09/29/26 11:38:11] [SSH] Starting sftp client.
[09/29/26 11:38:11] [SSH] Remote file system root /home/jenkins/agent does not exist.
Will try to create it...
[09/29/26 11:38:11] [SSH] Copying latest remoting.jar...
[09/29/26 11:38:11] [SSH] Copied 1,407,938 bytes.
Expanded the channel window size to 4MB
[09/29/26 11:38:11] [SSH] Starting agent process: cd "/home/jenkins/agent" && java -
jar remoting.jar -workDir /home/jenkins/agent -jar-cache
/home/jenkins/agent/remoting/jarCache
Sep 29, 2026 11:38:11 AM org.jenkinsci.remoting.engine.WorkDirManager
initializeWorkDir
INFO: Using /home/jenkins/agent/remoting as a remoting work directory
Sep 29, 2026 11:38:12 AM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to /home/jenkins/agent/remoting
<===[JENKINS REMOTING CAPACITY]==>channel started
Remoting version: 3355.3357.v931d3c992987
Launcher: SSHLauncher
Communication Protocol: Standard in/out
This is a Unix agent
Agent successfully connected and online
```

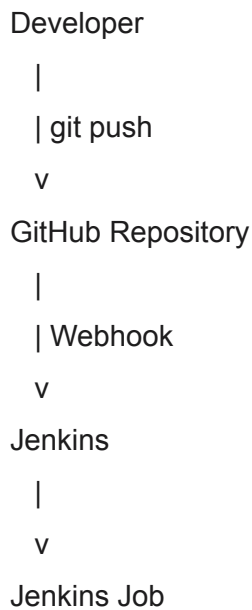
2. Configure webhooks to the Jenkins job.

Task 2 — Configure Webhook to Jenkins Job

Objective

Configure a GitHub webhook so that whenever code is pushed to the GitHub repository, Jenkins automatically triggers the configured job.

Flow:



Step 1 — Verify GitHub Plugin

Go to:

Manage Jenkins
→ Plugins
→ Installed Plugins

Verify:

GitHub Plugin

GitHub API Plugin

GitHub Branch Source Plugin

The required GitHub plugin was already installed and enabled.

Step 2 — Create Jenkins Job

Go to:

Jenkins Dashboard

→ New Item

Create:

Job Name:

webhook-job

Type:

Freestyle project

Step 3 — Configure Git Repository

Open:

webhook-job

→ Configure

→ Source Code Management

→ Git

Enter the GitHub repository URL:

<https://github.com/chandrashekar0532/Git-practice.git>

Configure the correct repository branch.

Example:

*/master

or:

*/main

depending on the repository branch.

Step 4 — Enable GitHub Trigger

Under:

Build Triggers

Enable:

☒ GitHub hook trigger for GITScm polling

Then:

Apply

→ Save

Step 5 — Create GitHub Webhook

Open:

GitHub Repository

→ Settings

→ Webhooks

→ Add webhook

Configure:

Payload URL:

http://<JENKINS-PUBLIC-IP>:<PORT>/github-webhook/

Content type:

application/json

Secret:

Blank for lab practice

Events:

Just the push event

Active:

Enabled

Then click:

Add webhook

Step 6 — Verify Webhook Connection

GitHub sent a ping request after creating the webhook.

The webhook page showed:

✓ Last delivery was successful

This confirmed that:

GitHub



Jenkins webhook endpoint

was successfully reachable.

Screenshot to add

Use your screenshot showing:

Last delivery was successful

Caption:

Figure 1: GitHub webhook successfully connected to the Jenkins server.

Step 7 — Test Automatic Build

Make a change in the Git repository:

```
echo "webhook test" >> webhook-test.txt  
git add webhook-test.txt  
git commit -m "Test Jenkins webhook"
```

```
git push origin <branch-name>
```

Do **not** click **Build Now**.

After pushing the code, verify that Jenkins automatically creates a new build.

Check:

```
Jenkins
→ webhook-job
→ Build History
→ Console Output
```

Successful flow should be:

```
Git Push
↓
GitHub
↓
Webhook
↓
Jenkins
↓
Job automatically triggered
```

Issue Faced — Wrong Git Remote / Branch

During testing, this command was used:

```
git push origin master
```

but the local branch was:

```
main
```

Git returned:

```
error: src refspec master does not match any
```

Also, the local repository was pointing to another GitHub repository:

```
git-challenges.git
```

instead of:

```
Git-practice.git
```

To verify the remote:

```
git remote -v
```

To correct it:

```
git remote set-url origin https://github.com/chandrashekar0532/Git-practice.git
```

Then push using the correct local branch.

For example:

```
git push origin main
```

Important Concept

A **webhook** is **event-driven**.

Without a webhook, Jenkins would need to repeatedly check GitHub for changes.

With a webhook:

Code pushed

→ GitHub immediately notifies Jenkins

→ Jenkins starts the job

This is commonly used in CI/CD automation.

Task 2 Final Result

GitHub Plugin verified	✓
Jenkins job configured	✓
Git repository configured	✓
GitHub trigger enabled	✓
Webhook created	✓
Webhook ping successful	✓
GitHub can communicate with Jenkins	✓
Automatic build test	

← → ↻ 🏠 Not secure 44.214.119.71:8081/job/webhook-job/configure ☆ 📄

📱 Apps 📺 YouTube 🌐 Google 📺 12 Beginner Python... 🐍 Learn Python 📘 Learn Data Structur... 🏆 LeetCode - The Wor... 🎬 Movierulz | Watch B... 🗨️ Slack

Jenkins / webhook-job / Configure 🔍 ⚙️

Configure

- ⚙️ General
- 🔗 Source Code Management**
- 🕒 Triggers
- 🌐 Environment
- 📋 Build Steps
- 🔧 Post-build Actions

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ **Git** ?

Repositories ?

Repository URL ?

https://github.com/chandrashekar0532/Git-practice.git

Credentials ?

- none - + Add

Advanced ▾

+ Add Repository

Save Apply

← → ↻ 🏠 github.com/chandrashekar0532/Git-practice/settings/hooks 🔍 📄

📱 Apps 📺 YouTube 🌐 Google 📺 12 Beginner Python... 🐍 Learn Python 📘 Learn Data Structur... 🏆 LeetCode - The Wor... 🎬 Movierulz | Watch B... 🗨️ Slack

☰ chandrashekar0532 / Git-practice 🔍 Type (⌘) to search 📄 + 🔍 📄 📄 📄

<> Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights **Settings**

Okay, that hook was successfully created. We sent a ping payload to test it out! Read more about it at <https://docs.github.com/webhooks/#ping-event>. ✕

Webhooks

Add webhook

Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our [Webhooks Guide](#).

✓ http://44.214.119.71:8081/github-w... (push)	Edit Delete
Last delivery was successful.	

- ⚙️ General
- Access
 - 👤 Collaborators
 - 🔒 Moderation ▾
- Code, planning, and automation
 - 📋 Rulesets ▾
 - 🔗 Branches
 - 🕒 Tags
 - 🔧 Actions ▾

MINGW64:/f/Git-practice

```
HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ git remote set-url origin https://github.com/chandrashekar0532/Git-practice.git
t

HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ git remote -v
origin https://github.com/chandrashekar0532/Git-practice.git (fetch)
origin https://github.com/chandrashekar0532/Git-practice.git (push)

HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ origin https://github.com/chandrashekar0532/Git-practice.git (fetch)
bash: syntax error near unexpected token `('

HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ origin https://github.com/chandrashekar0532/Git-practice.git (push)
bash: syntax error near unexpected token `('

HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ git push origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (13/13), 4.26 KiB | 1.06 MiB/s, done.
Total 13 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To https://github.com/chandrashekar0532/Git-practice.git
   224202a..09ae9ea  main -> main

HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ echo "webhook automatic test" >> webhook-auto-test.txt

HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ git add webhook-auto-test.txt
warning: in the working copy of 'webhook-auto-test.txt', LF will be replaced by CRLF the next
time git touches it

HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ git commit -m "Test automatic Jenkins webhook"
[main 49fb263] Test automatic Jenkins webhook
 1 file changed, 1 insertion(+)
 create mode 100644 webhook-auto-test.txt

HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 312 bytes | 312.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/chandrashekar0532/Git-practice.git
   09ae9ea..49fb263  main -> main

HP@DESKTOP-1GQO65F MINGW64 /f/Git-practice (main|SPARSE)
$ |
```

The screenshot shows the Jenkins web interface for a job named 'webhook-job'. The left sidebar contains a list of actions: Status, Changes, Workspace, Build Now, Delete Project, GitHub Hook Log, Configure, and Rename. The main area displays the job's status as 'webhook-job' with a green checkmark. Below this, there are 'Permalinks' for various build types: Last build (#4), Last stable build (#4), Last successful build (#4), Last failed build (#2), Last unsuccessful build (#2), and Last completed build (#4). A 'Builds' section on the left shows a list of builds with a filter input and a dropdown menu. The top of the page shows the Jenkins logo and the job name.

The screenshot shows the Jenkins console output for build #4 of the 'webhook-job'. The left sidebar contains a list of actions: Status, Changes, Console Output, Delete build '#4', Polling Log, Git Build Data, Edit build information, and Previous Build. The main area displays the console output, which includes the following text:

```
Started by GitHub push by chandrashekar0532
Running as SYSTEM
Building remotely on Jenkins-Agent-02 (agent02 linux) in workspace /home/jenkins/agent/workspace/webhook-job
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /home/jenkins/agent/workspace/webhook-job/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/chandrashekar0532/Git-practice.git # timeout=10
Fetching upstream changes from https://github.com/chandrashekar0532/Git-practice.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/chandrashekar0532/Git-practice.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 49fb2635eb4616051b1a91a4cb728670b6b51f8d (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 49fb2635eb4616051b1a91a4cb728670b6b51f8d # timeout=10
Commit message: "Test automatic Jenkins webhook"
First time build. Skipping changelog.
Finished: SUCCESS
```

3. Configure **Poll SCM** and **Build Periodically** options in the Jenkins job.

Task 3 — Configure Poll SCM and Build Periodically

Objective

Configure Jenkins job triggers using:

Poll SCM

Build Periodically

The purpose is to understand two different scheduled build methods in Jenkins.

Poll SCM

→ Jenkins checks the Git repository

→ Build runs only when code changes

Build Periodically

→ Jenkins runs the job on a schedule

→ Build runs even if code has not changed

Step 1 — Open Jenkins Job Configuration

Go to:

Jenkins Dashboard

→ webhook-job

→ Configure

Scroll to:

Build Triggers

Step 2 — Configure Poll SCM

Enable:

☒ Poll SCM

In the Schedule field, enter:

H/2 * * * *

Meaning:

Jenkins checks the Git repository approximately every 2 minutes.

Click:

Apply
→ Save

Step 3 — Disable Webhook Temporarily

For testing Poll SCM separately, temporarily disable:

GitHub hook trigger for GITScm polling

Keep only:

☒ Poll SCM

Reason:

If webhook remains enabled,
GitHub may trigger Jenkins immediately.

Then we cannot confirm whether Poll SCM triggered the build.

Step 4 — Create a Git Change

From the local Git repository:

```
echo "poll scm test" >> poll-scm-test.txt  
git add poll-scm-test.txt  
git commit -m "Test Poll SCM"  
git push origin main
```

Do not click:

Build Now

Wait for Jenkins to check the repository.

Step 5 — Verify Poll SCM

Open:

Jenkins
→ webhook-job

- Build History
- Latest Build
- Console Output

The console showed:

Started by an SCM change

It also showed:

Commit message: "Test Poll SCM"

Finished: SUCCESS

This confirmed:

- Jenkins checked GitHub
- detected a new commit
- automatically triggered the job

Screenshot to add

Use the console screenshot showing:

Started by an SCM change

Caption:

Figure 1: Jenkins job automatically triggered after Poll SCM detected a source-code change.

Configure Build Periodically

Step 6 — Enable Build Periodically

Go again to:

- webhook-job
- Configure
- Build Triggers

Enable:

☒ Build periodically

Enter:

H/2 * * * *

Keep Poll SCM configured as well.

Click:

Apply

→ Save

Step 7 — Test Without Changing Git Code

For this test:

Do not make any new Git commit

Do not push code

Do not click Build Now

Wait approximately 2 minutes.

Jenkins automatically started another build.

Step 8 — Verify Build Periodically

Open:

Build History

→ Latest Build

→ Console Output

The output showed:

Started by timer

and:

Finished: SUCCESS

This confirmed that Jenkins ran the job based only on the configured schedule.

Screenshot to add

Use your console screenshot showing:

Started by timer

and:

Finished: SUCCESS

Caption:

Figure 2: Jenkins job successfully triggered by the Build Periodically timer.

Cron Schedule Used

For both triggers:

H/2 * * * *

For this lab, it means Jenkins schedules the operation approximately every 2 minutes.

Basic Jenkins cron format:

MINUTE HOUR DAY-OF-MONTH MONTH DAY-OF-WEEK

Example:

H/5 * * * *

means approximately every 5 minutes.

H is Jenkins' hashed value, which helps distribute scheduled jobs instead of starting every Jenkins job at exactly the same second.

Important Difference

Poll SCM

Checks source-code repository
Runs only when change is detected
Trigger message:
Started by an SCM change

Build Periodically

Runs according to schedule
Code change is not required
Trigger message:
Started by timer

Simple Interview Answer

Poll SCM:

Poll SCM periodically checks the source-code repository. If Jenkins finds a new commit or code change, it triggers the job.

Build Periodically:

Build Periodically triggers a Jenkins job at a configured schedule regardless of whether the source code has changed.

Final Configuration

After testing, the Jenkins job can have:

- ☒ GitHub hook trigger for GITScm polling
- ☒ Poll SCM
- ☒ Build periodically

For the assignment, all three can remain configured.

In a real project, usually a webhook is preferred for immediate GitHub-based triggering, while frequent Poll SCM may be unnecessary because it repeatedly checks the repository.

Task 3 Final Result

Poll SCM configured	✓
Cron schedule configured	✓
Git change created	✓
SCM change detected	✓
Poll SCM build successful	✓
Build Periodically configured	✓
Timer-based build triggered	✓
Build Periodically successful	✓
Task 3 completed	✓

Final flow

Poll SCM:

Jenkins

|
| checks Git repository
v

GitHub

|
| new commit found
v

Jenkins Build

Build Periodically:

Jenkins Timer

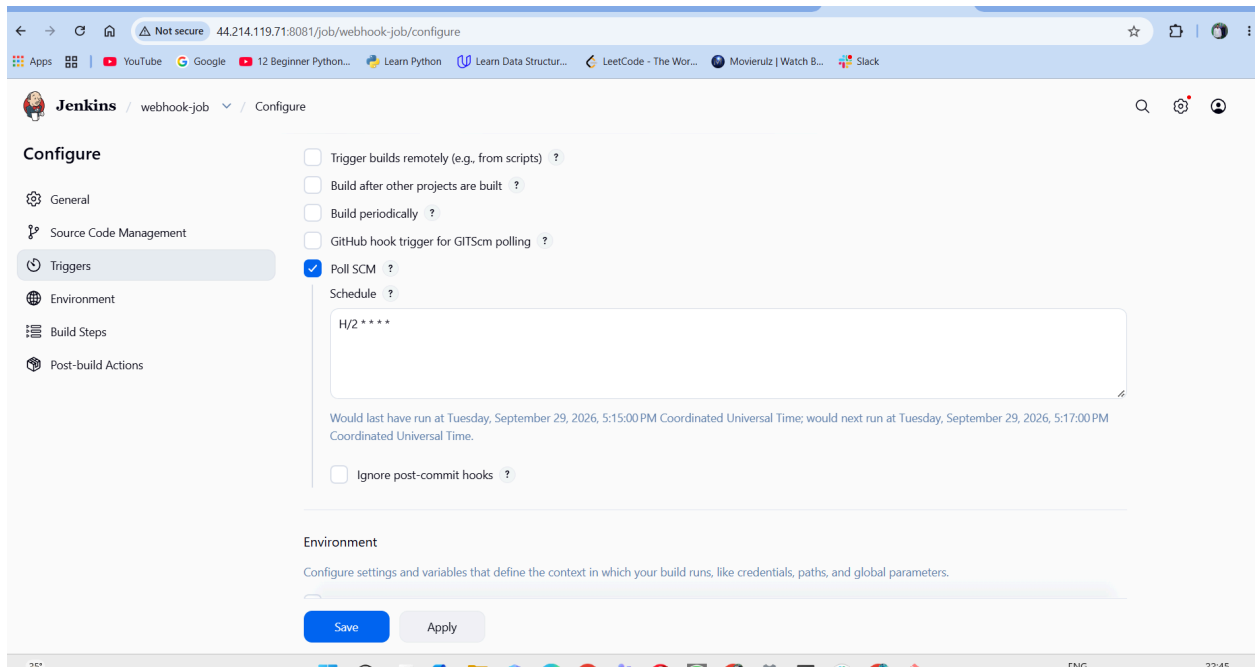
|
v

Jenkins Job

|

v

Build runs



The screenshot shows the Jenkins configuration page for a job named 'webhook-job'. The 'Configure' tab is active, and the 'Triggers' section is selected in the left sidebar. The 'Poll SCM' checkbox is checked, and the 'Schedule' field contains the cron expression 'H/2 * * * *'. Below the schedule field, a message indicates the last run was on Tuesday, September 29, 2026, at 5:15:00 PM, and the next run is on Tuesday, September 29, 2026, at 5:17:00 PM. The 'Environment' section is also visible, with a note about configuring settings and variables. The 'Save' and 'Apply' buttons are at the bottom.

Configure

- General
- Source Code Management
- Triggers**
- Environment
- Build Steps
- Post-build Actions

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☐ Build periodically ?

☐ GitHub hook trigger for GITScm polling ?

☒ Poll SCM ?

Schedule ?

H/2 * * * *

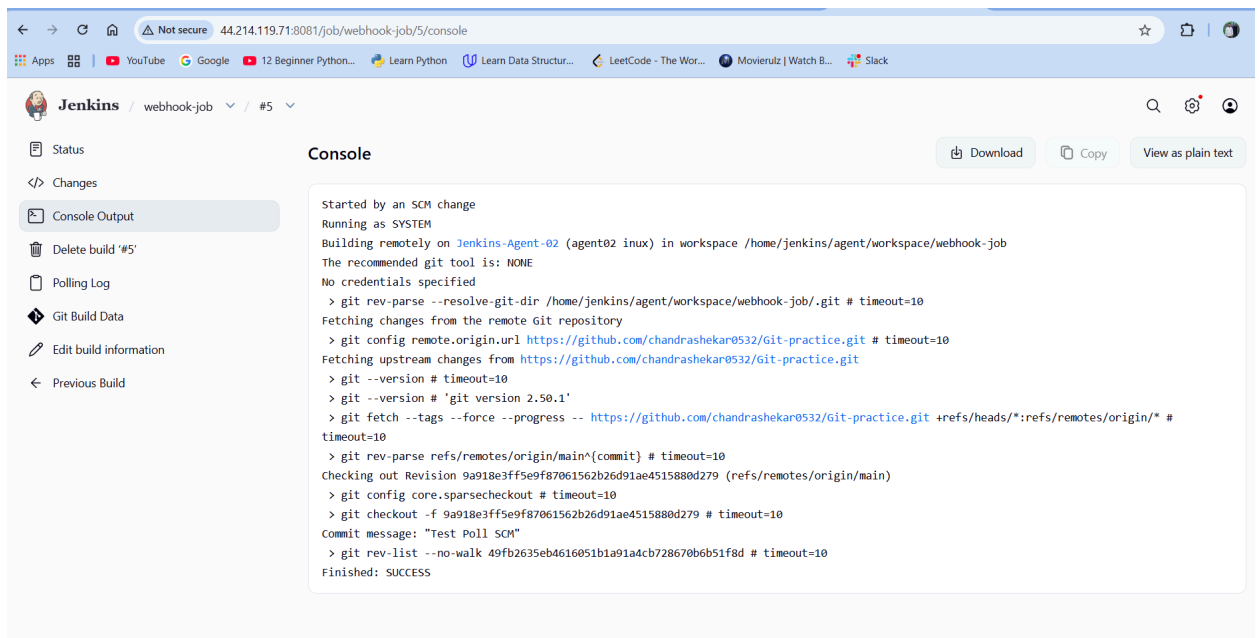
Would last have run at Tuesday, September 29, 2026, 5:15:00 PM Coordinated Universal Time; would next run at Tuesday, September 29, 2026, 5:17:00 PM Coordinated Universal Time.

☐ Ignore post-commit hooks ?

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

Save **Apply**

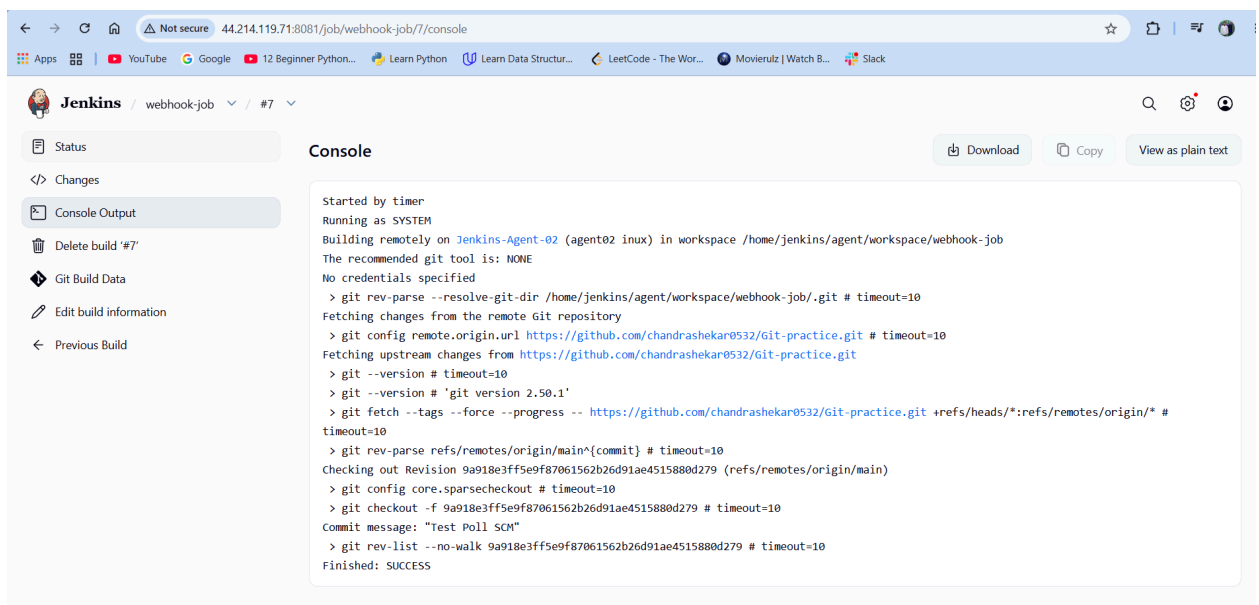
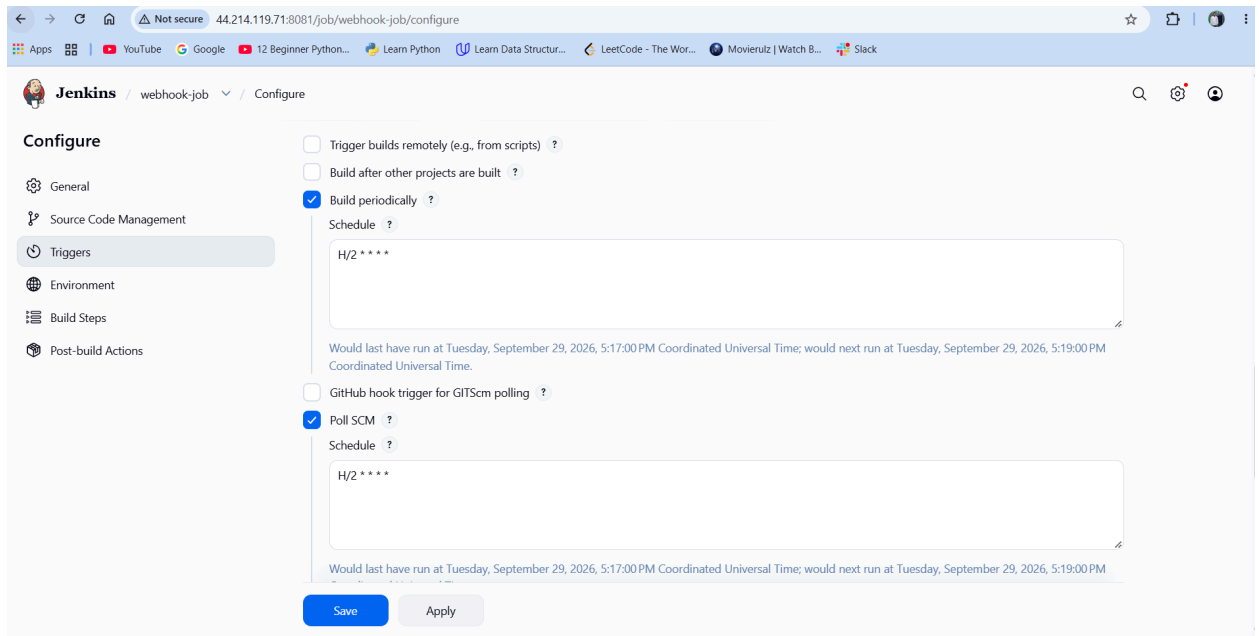


The screenshot shows the Jenkins console output for a build. The 'Console Output' tab is selected in the left sidebar. The console output shows the build process starting with a SCM change, running as SYSTEM, and building remotely on Jenkins-Agent-02. It then fetches changes from the remote Git repository, checks out the revision, and commits the changes. The build finishes successfully.

Console

Download **Copy** **View as plain text**

```
Started by an SCM change
Running as SYSTEM
Building remotely on Jenkins-Agent-02 (agent02 linux) in workspace /home/jenkins/agent/workspace/webhook-job
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /home/jenkins/agent/workspace/webhook-job/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/chandrashekar0532/Git-practice.git # timeout=10
Fetching upstream changes from https://github.com/chandrashekar0532/Git-practice.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/chandrashekar0532/Git-practice.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9a918e3ff5e9f87061562b26d91ae4515880d279 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9a918e3ff5e9f87061562b26d91ae4515880d279 # timeout=10
Commit message: "Test Poll SCM"
> git rev-list --no-walk 49fb2635eb4616051b1a91a4cb728670b6b51f8d # timeout=10
Finished: SUCCESS
```



4.Take a backup of the Jenkins server using a bash-script.

Task 4 — Take Jenkins Backup Using Bash Script

Objective

Take a backup of the Jenkins server using a Bash script by archiving the Jenkins home directory.

Jenkins home used:

```
/var/lib/jenkins
```

Backup location:

```
/opt/jenkins-backup
```

Step 1 — Verify Jenkins Home Directory

Run:

```
sudo ls -ld /var/lib/jenkins
```

Output:

```
drwxr-xr-x. 17 jenkins jenkins ... /var/lib/jenkins
```

This confirmed the Jenkins home directory exists.

Step 2 — Create Backup Directory

Run:

```
sudo mkdir -p /opt/jenkins-backup
```

Verify:

```
sudo ls -ld /opt/jenkins-backup
```

Output:

```
drwxr-xr-x. 2 root root ... /opt/jenkins-backup
```

Step 3 — Create Jenkins Backup Script

Create the script:

```
vi jenkins-backup.sh
```

Script:

```
#!/bin/bash
```

```
JENKINS_HOME="/var/lib/jenkins"
```

```
BACKUP_DIR="/opt/jenkins-backup"
```

```
DATE=$(date + %Y - %m - %d_%H - %M - %S)BACKUP_FILE = "
```

```
BACKUP_DIR/jenkins_backup_
```

```
DATE.tar.gz"echo"StoppingJenkins..."systemctlstopjenkinsecho"CreatingJenkinsbackup..."tar - czf
```

```
BACKUP_FILE" -C "$JENKINS_HOME".echo"Securingbackupfile..."chmod600"
```

```
BACKUP_FILE"
```

```
echo "Starting Jenkins..."
```

```
systemctl start jenkins
```

```
echo "Backup completed successfully."
```

```
echo "Backup file: $BACKUP_FILE"
```

Step 4 — Give Execute Permission

Run:

```
chmod +x jenkins-backup.sh
```

Verify:

```
ls -l jenkins-backup.sh
```

Output showed:

```
-rwxr-xr-x
```

This confirmed the script is executable.

Step 5 — Run the Backup Script

Run:

```
sudo ./jenkins-backup.sh
```

Output:

```
Stopping Jenkins...
Creating Jenkins backup...
Securing backup file...
Starting Jenkins...
Backup completed successfully.
Backup file: /opt/jenkins-backup/jenkins_backup_2026-09-30_06-46-42.tar.gz
```

Step 6 — Verify the Backup File

Run:

```
sudo ls -lh /opt/jenkins-backup/
```

Output:

```
jenkins_backup_2026-09-30_06-46-42.tar.gz
223M
```

This confirmed the Jenkins backup archive was successfully created.

Step 7 — Verify Backup Contents

Run:

```
sudo tar -tzf /opt/jenkins-backup/jenkins_backup_*.tar.gz | head
```

The archive contained Jenkins files such as:

```
secret.key
plugins/
.java/
```

This confirmed the backup archive is readable and contains Jenkins data.

Step 8 — Verify Jenkins Service

Run:

```
sudo systemctl status jenkins
```

Result:

Active: active (running)

The logs also showed:

Loaded all jobs

Completed initialization

Jenkins is fully up and running

This confirmed Jenkins restarted successfully after the backup.

Purpose of Task 4

The purpose is to protect Jenkins configuration and data.

The backup contains important Jenkins data such as:

Jobs

Plugins

Users

Nodes

Configuration

Build history

Secrets

Credentials-related data

The flow is:

Jenkins running



Stop Jenkins



Backup /var/lib/jenkins



Create tar.gz file



Start Jenkins



Verify service

Stopping Jenkins before the backup helps avoid copying files while Jenkins is actively changing them.

Issue Observed

You also had:

client_loop: send disconnect: Connection reset by peer

This happened to your SSH session, not to the backup itself.

You reconnected to the server and continued successfully, so it did not affect Task 4.

Task 4 Final Result

Jenkins home verified	✓
Backup directory created	✓
Bash script created	✓
Execute permission given	✓
Jenkins stopped by script	✓
Backup archive created	✓
Backup file verified	✓
Backup contents verified	✓
Jenkins restarted	✓
Jenkins running successfully	✓
Task 4 completed	✓

```
ec2-user@ip-172-31-12-117:~$
Last login: Wed Sep 30 06:23:56 2026 from 103.160.27.229
[ec2-user@ip-172-31-12-117 ~]$ sudo mkdir -p /opt/jenkins-backup
[ec2-user@ip-172-31-12-117 ~]$ sudo ls -ld /opt/jenkins-backup
drwxr-xr-x. 2 root root 6 Sep 30 06:42 /opt/jenkins-backup
[ec2-user@ip-172-31-12-117 ~]$ vi jenkins-backup.sh
[ec2-user@ip-172-31-12-117 ~]$ chmod +x jenkins-backup.sh
[ec2-user@ip-172-31-12-117 ~]$ ls -l jenkins-backup.sh
-rwxr-xr-x. 1 ec2-user ec2-user 476 Sep 30 06:43 jenkins-backup.sh
[ec2-user@ip-172-31-12-117 ~]$ sudo ./jenkins-backup.sh
Stopping Jenkins...
Creating Jenkins backup...
Securing backup file...
Starting Jenkins...
Backup completed successfully.
Backup file: /opt/jenkins-backup/jenkins_backup_2026-09-30_06-46-42.tar.gz
[ec2-user@ip-172-31-12-117 ~]$ sudo ls -lh /opt/jenkins-backup/
total 223M
-rw-r-----. 1 root root 223M Sep 30 06:46 jenkins_backup_2026-09-30_06-46-42.tar.gz
[ec2-user@ip-172-31-12-117 ~]$ sudo tar -tzf /opt/jenkins-backup/jenkins_backup_*.tar.gz | head
./
./java/
./java/fonts/
./java/fonts/21.0.12.1/
./java/fonts/21.0.12.1/fcinfo-1-ip-172-31-12-117.ec2.internal-amzn-2023-en-.properties
./secret.key
./secret.key.not-so-secret
./plugins/
./plugins/commons-lang3-api/
[ec2-user@ip-172-31-12-117 ~]$ sudo systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Drop-In: /etc/systemd/system/jenkins.service.d
            └─override.conf
   Active: active (running) since Wed 2026-09-30 06:47:05 UTC; 5min ago
   Main PID: 3592 (java)
   Tasks: 48 (limit: 1059)
   Memory: 317.4M
   CPU: 30.167s
   CGroup: /system.slice/jenkins.service
           └─3592 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8081

Sep 30 06:47:05 ip-172-31-12-117.ec2.internal jenkins[3592]: 2026-09-30 06:47:05.428+0000 [id=32] INFO hudson.slaves.SlaveComputer#tryReconnect: Atte
Sep 30 06:47:05 ip-172-31-12-117.ec2.internal jenkins[3592]: 2026-09-30 06:47:05.429+0000 [id=32] INFO hudson.slaves.SlaveComputer#tryReconnect: Atte
Sep 30 06:47:05 ip-172-31-12-117.ec2.internal jenkins[3592]: 2026-09-30 06:47:05.436+0000 [id=32] INFO jenkins.InitReactorRunner$1#onAttained: System
Sep 30 06:47:05 ip-172-31-12-117.ec2.internal jenkins[3592]: 2026-09-30 06:47:05.443+0000 [id=32] INFO jenkins.InitReactorRunner$1#onAttained: System
Sep 30 06:47:05 ip-172-31-12-117.ec2.internal jenkins[3592]: 2026-09-30 06:47:05.678+0000 [id=32] INFO jenkins.InitReactorRunner$1#onAttained: Loaded
Sep 30 06:47:05 ip-172-31-12-117.ec2.internal jenkins[3592]: 2026-09-30 06:47:05.681+0000 [id=32] INFO jenkins.InitReactorRunner$1#onAttained: Config
Sep 30 06:47:05 ip-172-31-12-117.ec2.internal jenkins[3592]: 2026-09-30 06:47:05.791+0000 [id=32] INFO jenkins.InitReactorRunner$1#onAttained: Comple
Sep 30 06:47:05 ip-172-31-12-117.ec2.internal jenkins[3592]: 2026-09-30 06:47:05.854+0000 [id=24] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is
Sep 30 06:47:12 ip-172-31-12-117.ec2.internal jenkins[3592]: 2026-09-30 06:47:12.976+0000 [id=90] INFO h.n.DiskSpaceMonitorDescriptor#markNodeOffline
lines 1-22/22 (END)
[1]~ Stopped
[ec2-user@ip-172-31-12-117 ~]$ sudo systemctl status jenkins
```

5. Take a backup of Jenkins using the **ThinBackup** plugin.

Task 5 — Take Jenkins Backup Using ThinBackup Plugin

Objective

Take a Jenkins backup using the **ThinBackup plugin** instead of using a custom Bash script.

The purpose is to use a Jenkins plugin that can create and manage backup sets from the Jenkins UI.

Step 1 — Install ThinBackup Plugin

Go to:

Manage Jenkins
→ Plugins
→ Available plugins

Search for:

ThinBackup

Install the plugin.

After installation, Jenkins showed:

ThinBackup Success
Loading plugin extensions Success

Screenshot to add

Use the plugin installation screenshot showing:

ThinBackup — Success

Caption:

Figure 1: ThinBackup plugin installed successfully in Jenkins.

Step 2 — Create Backup Directory

Connect to the Jenkins Controller using SSH.

Run:

```
sudo mkdir -p /opt/jenkins-thinbackup
```

Set Jenkins ownership:

```
sudo chown -R jenkins:jenkins /opt/jenkins-thinbackup
```

Set permissions:

```
sudo chmod 750 /opt/jenkins-thinbackup
```

Verify:

```
ls -ld /opt/jenkins-thinbackup
```

Output:

```
drwxr-x---. 2 jenkins jenkins ... /opt/jenkins-thinbackup
```

This confirmed that the Jenkins service user can write backup files to the directory.

Step 3 — Open ThinBackup Configuration

Go to:

Manage Jenkins
→ System

Find:

ThinBackup Configuration

The ThinBackup settings are integrated into Jenkins global configuration.

Step 4 — Configure Backup Directory

In:

Backup directory

Enter:

```
/opt/jenkins-thinbackup
```

For this practical, leave these fields blank:

Backup schedule for full backups

Backup schedule for differential backups

Keep:

Max number of backup sets:

-1

Then click:

Apply

→ Save

Screenshot to add

Use the screenshot showing:

ThinBackup Configuration

Backup directory:

/opt/jenkins-thinbackup

Caption:

Figure 2: ThinBackup backup directory configured in Jenkins global settings.

Step 5 — Open ThinBackup

Go to:

Manage Jenkins

→ ThinBackup

The page displayed:

Backup now

Restore

Step 6 — Run Manual Backup

Click:

Backup now

ThinBackup created a full backup set.

No Jenkins restart was required for this manual backup.

Step 7 — Verify Backup Directory

From the Jenkins server, run:

```
sudo ls -lh /opt/jenkins-thinbackup
```

Output:

```
total 16K
drwxr-xr-x. 5 jenkins jenkins 16K Sep 30 07:32 FULL-2026-09-30_07-32
```

The backup was created at:

```
/opt/jenkins-thinbackup/FULL-2026-09-30_07-32
```

This confirmed that ThinBackup successfully created a **FULL backup set**.

Screenshot to add

Use the terminal screenshot showing:

```
FULL-2026-09-30_07-32
```

Caption:

Figure 3: Jenkins full backup successfully created using the ThinBackup plugin.

Step 8 — Verify Backup Contents

Run:

```
sudo ls -lah /opt/jenkins-thinbackup/FULL-2026-09-30_07-32
```

This verifies that Jenkins configuration and job-related backup files were created inside the backup set.

You can also use:

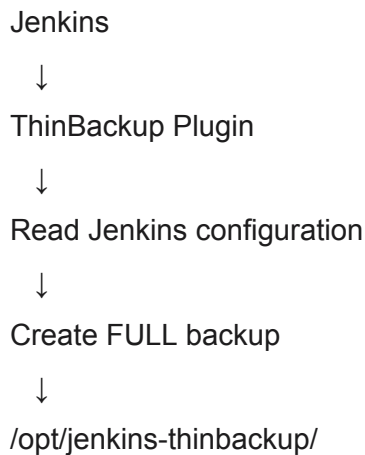
```
sudo find /opt/jenkins-thinbackup/FULL-2026-09-30_07-32 -maxdepth 2 -type f | head
-20
```

to view some of the files included in the backup.

Purpose of ThinBackup

ThinBackup provides Jenkins backup functionality directly through Jenkins.

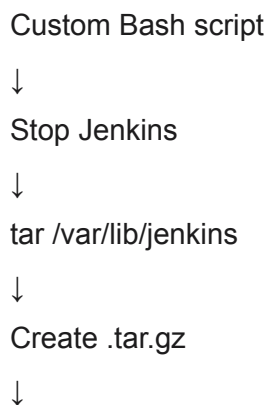
Basic flow:



It can also be configured for scheduled full and differential backups.

Task 4 vs Task 5

Task 4 — Bash Backup



Start Jenkins

Whereas:

Task 5 — ThinBackup

Jenkins UI

↓

ThinBackup Plugin

↓

Backup now

↓

FULL backup directory created

Important difference

Bash backup gives you direct control over the entire Jenkins home backup.

ThinBackup provides plugin-based Jenkins configuration backup and easier backup/restore management through the Jenkins UI.

Interview Answer

If asked “**How did you take Jenkins backup using ThinBackup?**”, you can say:

I installed the ThinBackup plugin, created a backup directory with Jenkins user permissions, configured that directory under Jenkins global settings, and then triggered a full backup using the Backup Now option. I verified the backup set on the Jenkins server under `/opt/jenkins-thinbackup`.

Task 5 Final Result

ThinBackup plugin installed 

Backup directory created 

Jenkins ownership configured 

ThinBackup path configured	✓
Manual backup triggered	✓
FULL backup created	✓
Backup directory verified	✓
Task 5 completed	✓

Your actual backup proof is:

/opt/jenkins-thinbackup/FULL-2026-09-30_07-32

So **Task 5** is fully completed. ✓

The screenshot shows the Jenkins web interface at the URL `44.201.6.78:8081/manage/pluginManager/updates/`. The browser's address bar indicates a "Not secure" connection. The page title is "Jenkins / Manage Jenkins / Plugins". On the left, a sidebar menu lists "Updates", "Available plugins", "Installed plugins", "Advanced settings", and "Download progress", with "Download progress" currently selected. The main content area is titled "Download progress" and shows the status of the "ThinBackup" plugin installation. The "Preparation" step is completed, with a list of actions: "Checking internet connectivity", "Checking update center connectivity", and "Success". The "ThinBackup" plugin itself is marked as "Success" with a green checkmark. Below this, "Loading plugin extensions" is also marked as "Success" with a green checkmark. At the bottom, there are two instructions: a link to "Go back to the top page" (noted as "you can start using the installed plugins right away") and a checkbox for "Restart Jenkins when installation is complete and no jobs are running", which is currently unchecked.

← → ↻ 🏠 ⚠ Not secure 44.201.6.78:8081/manage/pluginManager/updates/

Apps | YouTube | Google | 12 Beginner Python... | Learn Python | Learn Data Structur... | LeetCode - The Wor... | Movierulz | Watch B... | Slack

Jenkins / Manage Jenkins / Plugins

Plugins

- Updates
- Available plugins
- Installed plugins
- Advanced settings
- Download progress**

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

ThinBackup ✓ Success

Loading plugin extensions ✓ Success

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

→ ☐ Restart Jenkins when installation is complete and no jobs are running


```
ec2-user@ip-172-31-12-117:~$
ast login: Wed Sep 30 07:14:50 2026 From 103.160.27.229
ec2-user@ip-172-31-12-117 ~]$ sudo ls -lh /opt/jenkins-thinbackup
total 16K
-rwxr-xr-x. 5 jenkins jenkins 16K Sep 30 07:32 FULL-2026-09-30_07-32
ec2-user@ip-172-31-12-117 ~]$ sudo ls -lah /opt/jenkins-thinbackup/FULL-2026-09-30_07-32
total 204K
-rwxr-xr-x. 5 jenkins jenkins 16K Sep 30 07:32 .
-rwxr-xr-x. 3 jenkins jenkins 35 Sep 30 07:32 ..
-rw-r--r--. 1 jenkins jenkins 0 Sep 30 07:32 backup-completed.info
-rw-r--r--. 1 jenkins jenkins 272 Sep 30 07:21 com.cloudbees.hudson.plugins.folder.config.AbstractFolderConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 3.4K Sep 30 07:21 config.xml
-rw-r--r--. 1 jenkins jenkins 4.0K Sep 29 10:33 credentials.xml
-rw-r--r--. 1 jenkins jenkins 177 Sep 30 07:21 github-plugin-configuration.xml
-rw-r--r--. 1 jenkins jenkins 156 Sep 30 06:47 hudson.model.UpdateCenter.xml
-rw-r--r--. 1 jenkins jenkins 238 Sep 30 07:21 hudson.plugins.build_timeout.global.GlobalTimeoutConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 250 Sep 30 07:21 hudson.plugins.build_timeout.operations.BuildStepOperation.xml
-rw-r--r--. 1 jenkins jenkins 1.5K Sep 30 07:21 hudson.plugins.email_ext.ExtendedEmailPublisher.xml
-rw-r--r--. 1 jenkins jenkins 630 Sep 30 07:21 hudson.plugins.git.GitSCM.xml
-rw-r--r--. 1 jenkins jenkins 370 Sep 28 10:53 hudson.plugins.git.GitTool.xml
-rw-r--r--. 1 jenkins jenkins 504 Sep 30 07:21 hudson.plugins.gradle.enriched.EnrichedSummaryConfig.xml
-rw-r--r--. 1 jenkins jenkins 654 Sep 30 07:21 hudson.plugins.gradle.injection.InjectionConfig.xml
-rw-r--r--. 1 jenkins jenkins 382 Sep 30 07:21 hudson.plugins.timestamp.TimestamperConfig.xml
-rw-r--r--. 1 jenkins jenkins 261 Sep 30 07:21 hudson.tasks.Mailer.xml
-rw-r--r--. 1 jenkins jenkins 76 Sep 30 07:21 hudson.tasks.Shell.xml
-rw-r--r--. 1 jenkins jenkins 216 Sep 30 07:21 hudson.triggers.SCMTrigger.xml
-rw-r--r--. 1 jenkins jenkins 11K Sep 30 07:32 installedPlugins.xml
-rw-r--r--. 1 jenkins jenkins 297 Sep 30 07:21 io.jenkins.plugins.junit.storage.JunitTestResultStorageConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 307 Sep 28 11:52 io.jenkins.plugins.prism.PrismConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 277 Sep 30 07:21 jenkins.fingerprints.GlobalFingerprintConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 159 Sep 30 07:21 jenkins.model.ArtifactManagerConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 253 Sep 30 07:21 jenkins.model.GlobalBuildDiscarderConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 241 Sep 30 07:21 jenkins.model.GlobalComputerRetentionCheckIntervalConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 265 Sep 30 07:21 jenkins.model.JenkinsLocationConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 241 Sep 28 11:52 jenkins.plugins.git.GitHooksConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 169 Sep 28 11:52 jenkins.security.QueueItemAuthenticatorConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 86 Sep 30 07:21 jenkins.security.ResourceDomainConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 162 Sep 28 11:52 jenkins.security.UpdateSiteWarningsConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 357 Sep 28 11:52 jenkins.security.apitoken.ApiTokenPropertyConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 179 Sep 30 07:21 jenkins.tasks.filters.EnvVarsFilterGlobalConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 171 Sep 28 10:22 jenkins.telemetry.Correlator.xml
-rwxr-xr-x. 8 jenkins jenkins 147 Sep 30 07:32 jobs
-rw-r--r--. 1 jenkins jenkins 1.1K Sep 30 06:47 nodeMonitors.xml
-rwxr-xr-x. 4 jenkins jenkins 54 Sep 29 11:23 nodes
-rw-r--r--. 1 jenkins jenkins 170 Sep 30 07:21 org.jenkinsci.plugins.displayurlapi.DefaultDisplayURLProviderGlobalConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 325 Sep 28 11:52 org.jenkinsci.plugins.gitclient.GitHostKeyVerificationConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 289 Sep 30 07:21 org.jenkinsci.plugins.github_branch_source.GitHubConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 169 Sep 30 07:21 org.jenkinsci.plugins.workflow.flow.GlobalDefaultFlowDurabilityLevel.xml
-rw-r--r--. 1 jenkins jenkins 229 Sep 30 07:21 org.jenkinsci.plugins.workflow.libs.GlobalLibraries.xml
-rw-r--r--. 1 jenkins jenkins 247 Sep 30 07:21 org.jenkinsci.plugins.workflow.libs.GlobalUntrustedLibraries.xml
-rw-r--r--. 1 jenkins jenkins 1.1K Sep 30 07:21 org.jvnet.hudson.plugins.thinbackup.ThinBackupPluginImpl.xml
-rw-r--r--. 1 jenkins jenkins 259 Sep 30 07:32 queue.xml
-rw-r--r--. 1 jenkins jenkins 334 Sep 28 11:52 scriptApproval.xml
-rwxr-xr-x. 6 jenkins jenkins 16K Sep 28 12:42 users
ec2-user@ip-172-31-12-117 ~]$
```

6.Set up a new Jenkins server and restore the backup taken in task 4.

Task 6 — Set Up a New Jenkins Server and Restore Backup

Objective

Create a new Jenkins server and restore the Jenkins backup created in Task 4.

Backup file used:

jenkins_backup_2026-09-30_06-46-42.tar.gz

Backup size:

223 MB

The restore should recover:

Jobs
Plugins
Credentials
Users
Nodes
Build history
Jenkins configuration

Architecture

```
Old Jenkins Server
44.201.6.78
|
| Task 4 backup
v
jenkins_backup_2026-09-30_06-46-42.tar.gz
|
| SCP
v
Local Machine
|
| SCP
v
New Jenkins Restore Server
54.89.165.175
|
v
Restore /var/lib/jenkins
```

Step 1 — Launch New EC2 Instance

Create a new EC2 instance.

Configuration:

Name:
Jenkins-Restore-Server

AMI:
Amazon Linux 2023

Instance Type:
t3.micro

Key Pair:
my_server

VPC:
Same VPC

Public IP:
Enabled

Allow:

SSH Port 22
Jenkins Port 8081

New restore server public IP:

54.89.165.175

Screenshot

Use the AWS screenshot showing:

Jenkins-Restore-Server
Running

Caption:

Figure 1: New EC2 instance created for Jenkins backup restoration.

Step 2 — Connect to New Restore Server

From Git Bash:

```
chmod 400 "my_server.pem"
```

Connect:

```
ssh -i "my_server.pem" ec2-user@ec2-54-89-165-175.compute-1.amazonaws.com
```

Successful prompt:

```
[ec2-user@ip-172-31-20-166 ~]$
```

Step 3 — Install Java 21

Run:

```
sudo dnf install java-21-amazon-corretto-devel -y
```

Verify:

```
java -version
```

Output:

```
openjdk version "21.0.12.1"  
Amazon Corretto
```

Jenkins requires Java to run.

Step 4 — Configure Jenkins Repository

Add Jenkins stable repository:

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \  
https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

Import Jenkins signing key:

```
sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
```

Check available Jenkins versions:

```
sudo dnf --showduplicates list jenkins
```

The old Jenkins server was using:

2.568.3

The same version was available on the new server.

Step 5 — Install Same Jenkins Version

Install:

```
sudo dnf install jenkins-2.568.3-1 -y
```

Verify:

```
jenkins --version
```

Output:

2.568.3

Using the same Jenkins version reduces compatibility problems during restore.

Step 6 — Locate Task 4 Backup on Old Jenkins Server

Connect to the old Jenkins server.

Verify backup:

```
sudo ls -lh /opt/jenkins-backup/
```

Output:

```
223M jenkins_backup_2026-09-30_06-46-42.tar.gz
```

Original backup location:

```
/opt/jenkins-backup/jenkins_backup_2026-09-30_06-46-42.tar.gz
```

Step 7 — Copy Backup to ec2-user Home

Because the backup was owned by root, copy it to the `ec2-user` home directory.

Run:

```
sudo cp \  
/opt/jenkins-backup/jenkins_backup_2026-09-30_06-46-42.tar.gz \  
/home/ec2-user/
```

Change ownership:

```
sudo chown ec2-user:ec2-user \  
/home/ec2-user/jenkins_backup_2026-09-30_06-46-42.tar.gz
```

Verify:

```
ls -lh /home/ec2-user/jenkins_backup_2026-09-30_06-46-42.tar.gz
```

Output:

```
223M  
ec2-user ec2-user
```

Step 8 — Download Backup to Local Machine

Exit from old server.

From local Git Bash:

```
scp -i "my_server.pem" \  
ec2-user@ec2-44-201-6-78.compute-1.amazonaws.com:/home/ec2-user/jenkins_backup_2026-09-30_06-46-42.tar.gz .
```

Transfer completed successfully:

100% 222MB

Verify locally:

```
ls -lh jenkins_backup_2026-09-30_06-46-42.tar.gz
```

Output:

223M

Step 9 — Upload Backup to New Restore Server

From local Git Bash:

```
scp -i "my_server.pem" \  
jenkins_backup_2026-09-30_06-46-42.tar.gz \  
ec2-user@ec2-54-89-165-175.compute-1.amazonaws.com:/home/ec2-user/
```

Connect to the new restore server:

```
ssh -i "my_server.pem" \  
ec2-user@ec2-54-89-165-175.compute-1.amazonaws.com
```

Verify:

```
ls -lh /home/ec2-user/jenkins_backup_2026-09-30_06-46-42.tar.gz
```

Output:

223M

Step 10 — Stop Jenkins Before Restore

Run:

```
sudo systemctl stop jenkins
```

Check the new Jenkins home:

```
sudo ls -la /var/lib/jenkins | head
```

Initially it was nearly empty.

Step 11 — Restore Jenkins Backup

Extract the Task 4 backup into Jenkins home:

```
sudo tar -xzf \  
/home/ec2-user/jenkins_backup_2026-09-30_06-46-42.tar.gz \  
-C /var/lib/jenkins
```

Fix ownership:

```
sudo chown -R jenkins:jenkins /var/lib/jenkins
```

Verify:

```
sudo ls -la /var/lib/jenkins | head -20
```

Restored files included:

```
config.xml  
credentials.xml  
plugins/  
jobs/  
users/  
secrets/  
nodes/
```

This confirmed the Jenkins home backup was restored.

Step 12 — Configure Jenkins Port 8081

The old Jenkins server was using port:

8081

Create systemd override directory:

```
sudo mkdir -p /etc/systemd/system/jenkins.service.d
```

Create:

```
sudo vi /etc/systemd/system/jenkins.service.d/override.conf
```

Add:

```
[Service]  
Environment="JENKINS_PORT=8081"
```

Reload systemd:

```
sudo systemctl daemon-reload
```

Verify:

```
sudo systemctl cat jenkins | tail -10
```

Output included:

```
Environment="JENKINS_PORT=8081"
```

Step 13 — Start Jenkins

Start:

```
sudo systemctl start jenkins
```

Check status:

```
sudo systemctl status jenkins
```

Result:

```
Active: active (running)
```

This confirmed Jenkins started successfully using the restored Jenkins home.

Step 14 — Enable Jenkins After Reboot

Initially Jenkins showed:

```
disabled
```

Enable it:

```
sudo systemctl enable jenkins
```

Verify:

```
sudo systemctl is-enabled jenkins
```

Output:

```
enabled
```

Now Jenkins will start automatically after an EC2 reboot.

Step 15 — Open Restored Jenkins Server

Open:

```
http://54.89.165.175:8081
```

The restored Jenkins dashboard opened successfully.

The following old jobs were visible:

```
Label-Test-Job  
LoginApp-Build  
LoginApp-Checkout  
LoginApp-Package  
Private-Repo-Checkout  
webhook-job
```

Old nodes were also restored:

```
Jenkins-Agent-01  
Jenkins-Agent-02
```

Build history was also visible.

Screenshot

Use your final Jenkins dashboard screenshot showing the jobs and both agents.

Caption:

Figure 2: Jenkins jobs, agents, and build history successfully restored on the new Jenkins server.

Restore Verification

The following data was successfully restored:

```
Jenkins jobs  
Build history  
Jenkins configuration  
Plugins  
Credentials configuration  
Users  
Agent configuration  
Jenkins port configuration
```



Jenkins service running
Jenkins enabled after reboot



Issues Faced

Issue 1 — Backup Not Found

Initially this command was attempted on the new server:

```
sudo cp /opt/jenkins-backup/jenkins_backup_2026-09-30_06-46-42.tar.gz ...
```

Error:

```
No such file or directory
```

Reason

The backup existed on the old Jenkins server, not the new restore server.

Fix

Connected to the old Jenkins server, copied the backup to `ec2-user`, downloaded it locally, and uploaded it to the new server.

Issue 2 — SCP Executed from Wrong Machine

The upload command was initially executed after SSH login to the new server.

Error:

```
scp: stat local ... No such file or directory
```

Reason

The backup file was stored on the Windows local machine.

Fix

Exited the EC2 SSH session and executed `scp` from local Git Bash.

Issue 3 — Jenkins Service Disabled

Jenkins was running, but status showed:

```
disabled
```

Fix

Run:

```
sudo systemctl enable jenkins
```

Verified:

```
enabled
```

Purpose of Task 6

The purpose of this task is to learn Jenkins disaster recovery and migration.

If the original Jenkins server fails, a backup of `JENKINS_HOME` can be restored onto another Jenkins server.

Flow:

```
Jenkins Server Failure
      ↓
Create New Server
      ↓
Install Java
      ↓
Install Same Jenkins Version
      ↓
Copy Backup
      ↓
Restore /var/lib/jenkins
      ↓
Fix Ownership
      ↓
Start Jenkins
      ↓
Jobs and Configuration Recovered
```

Simple Interview Answer

I created a new EC2 Jenkins server, installed the same Java and Jenkins version as the source server, transferred the Jenkins home backup, stopped Jenkins, extracted the backup into `/var/lib/jenkins`, corrected ownership using `jenkins:jenkins`, configured the required Jenkins port, and started Jenkins. I verified that the jobs, agents, plugins, credentials configuration, and build history were restored successfully.

```
ec2-user@ip-172-31-20-166:~  
Last login: Wed Sep 30 09:08:26 2026 from 103.160.27.229  
[ec2-user@ip-172-31-20-166 ~]$ ls -lh /home/ec2-user/jenkins_backup_2026-09-30_06-46-42.tar.gz  
-rw-r--r--. 1 ec2-user ec2-user 223M Sep 30 09:12 /home/ec2-user/jenkins_backup_2026-09-30_06-46-42.tar.gz  
[ec2-user@ip-172-31-20-166 ~]$ sudo systemctl stop jenkins  
[ec2-user@ip-172-31-20-166 ~]$ sudo ls -la /var/lib/jenkins | head  
total 16  
drwxr-xr-x. 2 jenkins jenkins 6 Sep 30 08:10 .  
drwxr-xr-x. 27 root root 16384 Sep 30 08:10 ..  
[ec2-user@ip-172-31-20-166 ~]$ sudo tar -xzf /home/ec2-user/jenkins_backup_2026-09-30_06-46-42.tar.gz \  
-C /var/lib/jenkins  
[ec2-user@ip-172-31-20-166 ~]$ sudo chown -R jenkins:jenkins /var/lib/jenkins  
[ec2-user@ip-172-31-20-166 ~]$ sudo ls -la /var/lib/jenkins | head -20  
total 168  
drwxr-xr-x. 17 jenkins jenkins 16384 Sep 30 06:46 .  
drwxr-xr-x. 27 root root 16384 Sep 30 08:10 ..  
-rw-----. 1 jenkins jenkins 75 Sep 29 12:00 .bash_history  
drwxr-xr-x. 3 jenkins jenkins 17 Sep 28 10:22 .cache  
drwxr-xr-x. 3 jenkins jenkins 18 Sep 28 16:03 .config  
drwxr-xr-x. 3 jenkins jenkins 20 Sep 28 10:53 .groovy  
drwxr-xr-x. 3 jenkins jenkins 19 Sep 28 10:22 .java  
-rw-r--r--. 1 jenkins jenkins 0 Sep 30 06:13 .lastStarted  
drwxr-xr-x. 3 jenkins jenkins 24 Sep 28 16:12 .m2  
-rw-r--r--. 1 jenkins jenkins 4 Sep 29 16:53 .owner  
-rw-r--r--. 1 jenkins jenkins 3803 Sep 30 06:13 config.xml  
-rw-r--r--. 1 jenkins jenkins 4075 Sep 29 10:33 credentials.xml  
drwxr-xr-x. 4 jenkins jenkins 26 Sep 29 10:39 fingerprints  
-rw-r--r--. 1 jenkins jenkins 156 Sep 30 06:13 hudson.model.UpdateCenter.xml  
-rw-r--r--. 1 jenkins jenkins 370 Sep 28 10:53 hudson.plugins.git.GitTool.xml  
-rw-----. 1 jenkins jenkins 1680 Sep 28 10:53 identity.key.enc  
-rw-r--r--. 1 jenkins jenkins 307 Sep 28 11:52 io.jenkins.plugins.prism.Prism  
Configuration.xml  
-rw-r--r--. 1 jenkins jenkins 7 Sep 30 06:13 jenkins.install.InstallUtil.la  
stExecVersion  
-rw-r--r--. 1 jenkins jenkins 7 Sep 28 10:59 jenkins.install.UpgradeWizard.  
state  
[ec2-user@ip-172-31-20-166 ~]$ sudo mkdir -p /etc/systemd/system/jenkins.service  
.d  
[ec2-user@ip-172-31-20-166 ~]$ sudo vi /etc/systemd/system/jenkins.service.d/ove  
rride.conf  
[ec2-user@ip-172-31-20-166 ~]$ sudo systemctl daemon-reload  
[ec2-user@ip-172-31-20-166 ~]$ sudo systemctl cat jenkins | tail -10  
# If unset, the value from the OS is inherited, which is normally 0022.  
# The default umask comes from pam_umask(8) and /etc/login.defs.  
#UMask=0022  
  
[Install]  
WantedBy=multi-user.target  
  
# /etc/systemd/system/jenkins.service.d/override.conf  
[Service]  
Environment="JENKINS_PORT=8081"  
[ec2-user@ip-172-31-20-166 ~]$ sudo systemctl start jenkins  
[ec2-user@ip-172-31-20-166 ~]$ sudo systemctl status jenkins
```

[illegible]

The screenshot shows the Jenkins web interface. On the left sidebar, there are options: '+ New Item', 'Build History', 'Project Relationship', and 'Check File Fingerprint'. Below these are two dropdown menus: 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status' (showing 'Built-In Node' with 0/1, 'Jenkins-Agent-01' with 0/1, and 'Jenkins-Agent-02' with 0/1). The main area displays a table of builds with the following columns: S (Status), W (Webhook), Name, Last Success, Last Failure, and Last Duration. The builds listed are: Label-Test-Job (1 day 19 hr, #4, 51 ms), LoginApp-Build (1 day 17 hr, #1, 25 sec), LoginApp-Checkout (1 day 17 hr, #1, 3.8 sec), LoginApp-Package (1 day 16 hr, #2, 18 sec), Private-Repo-Checkout (1 day 15 hr, #1, 1.1 sec), and webhook-job (25 sec, #28, 16 hr, #2, 3.2 sec). All builds show a green checkmark icon, indicating they are successful.

Task 6 Final Result

New EC2 server created
 Java 21 installed
 Jenkins 2.568.3 installed
 Task 4 backup located
 Backup transferred to local machine
 Backup uploaded to restore server
 Jenkins stopped before restore
 Backup extracted to /var/lib/jenkins
 Ownership corrected
 Port 8081 configured
 Jenkins started successfully
 Jenkins enabled at boot
 Old jobs restored
 Old agents restored
 Build history restored
 Task 6 completed



Task 6 — Successfully Completed.